



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks

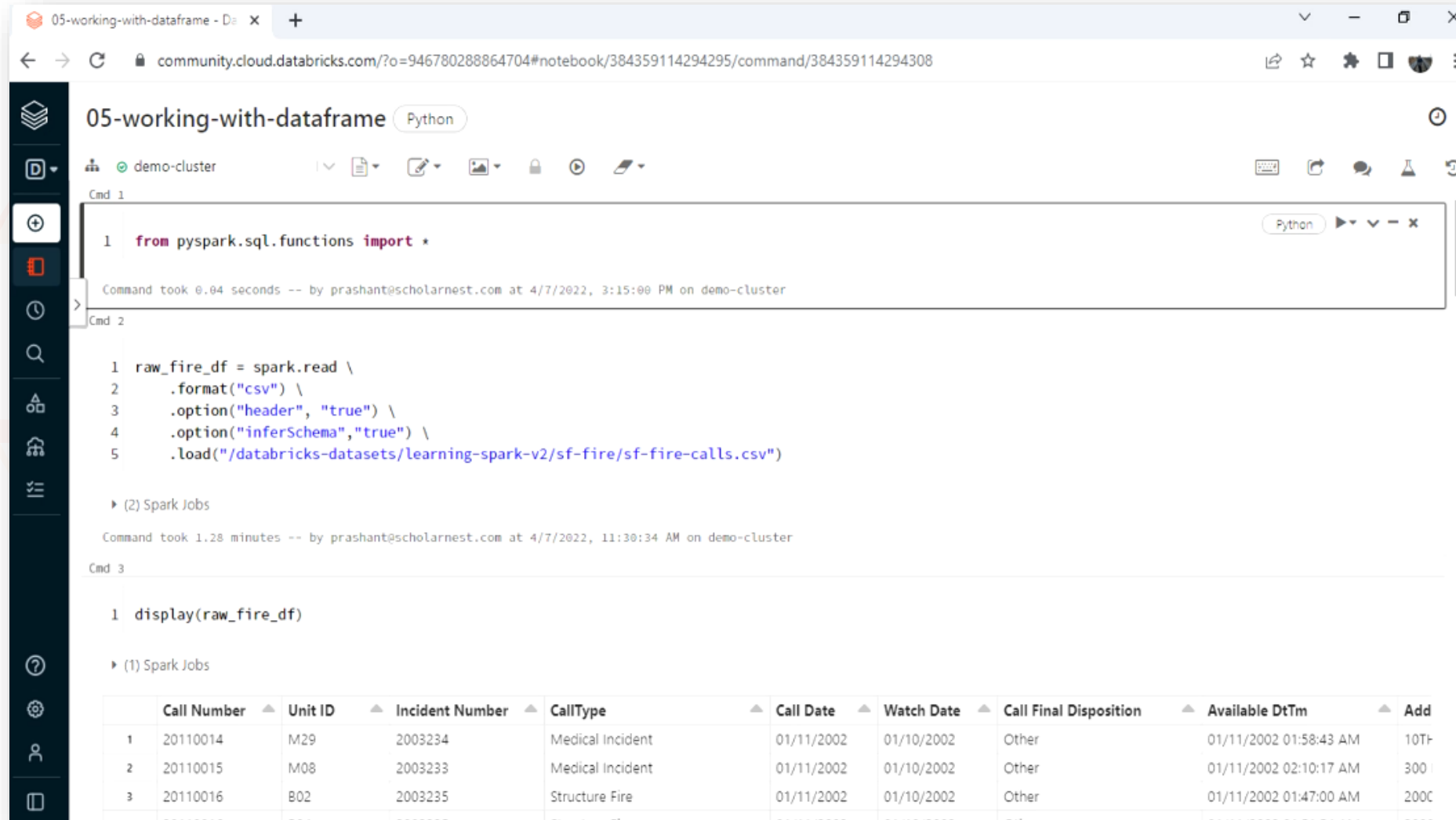


Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Working with Spark Dataframe – Part II

Go to your Databricks workspace and open the notebook we worked with in the previous lecture. **(Reference: 05-working-with-dataframe.ipynb)**



05-working-with-dataframe - Python

demo-cluster

Cmd 1

```
1 from pyspark.sql.functions import *
```

Command took 0.04 seconds -- by prashant@scholarnest.com at 4/7/2022, 3:15:00 PM on demo-cluster

Cmd 2

```
1 raw_fire_df = spark.read \  
2   .format("csv") \  
3   .option("header", "true") \  
4   .option("inferSchema", "true") \  
5   .load("/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv")
```

► (2) Spark Jobs

Command took 1.28 minutes -- by prashant@scholarnest.com at 4/7/2022, 11:30:34 AM on demo-cluster

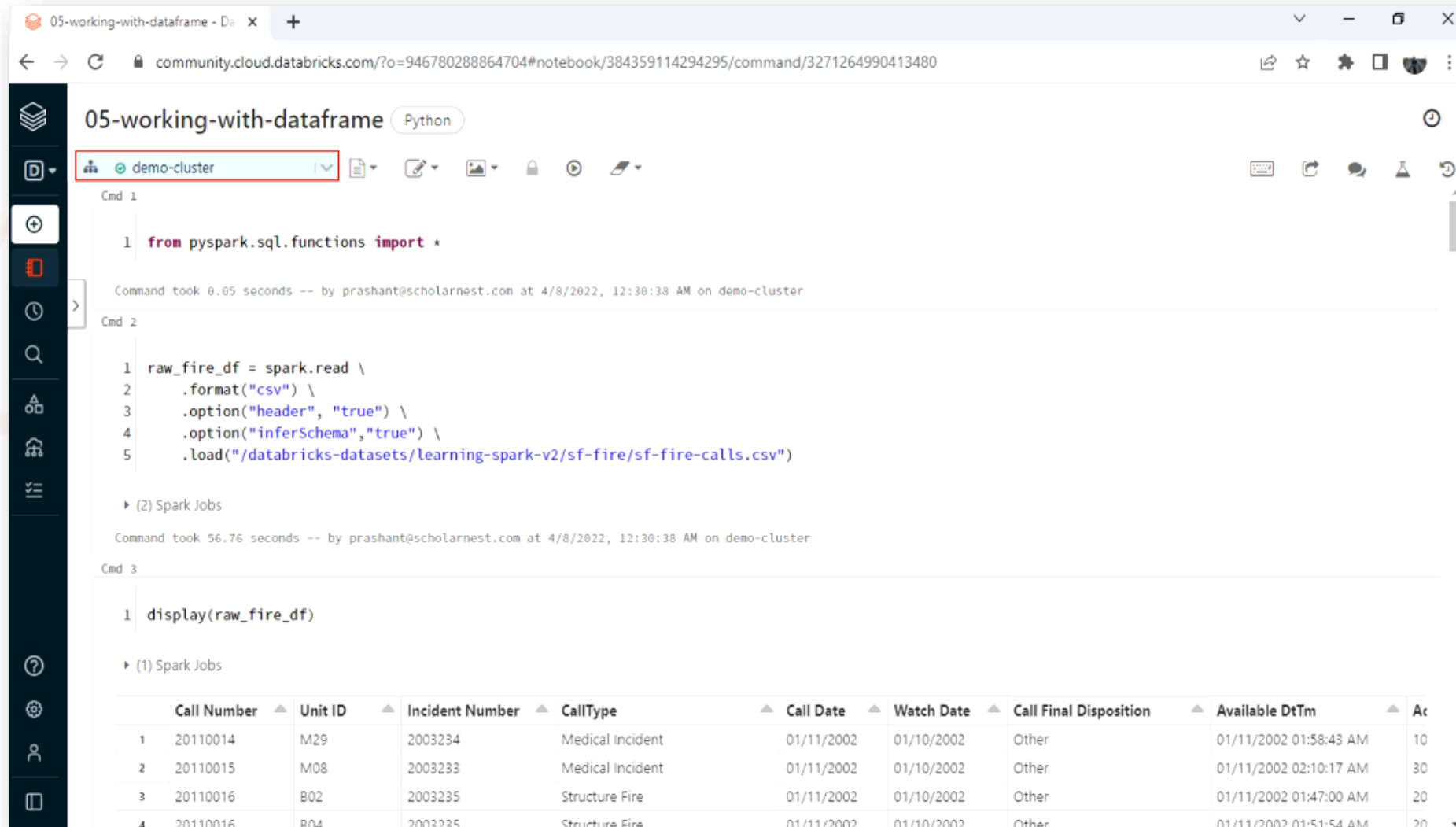
Cmd 3

```
1 display(raw_fire_df)
```

► (1) Spark Jobs

	Call Number	Unit ID	Incident Number	CallType	Call Date	Watch Date	Call Final Disposition	Available DtTm	Add
1	20110014	M29	2003234	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 01:58:43 AM	10TH
2	20110015	M08	2003233	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 02:10:17 AM	300
3	20110016	B02	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	200C
4	20110016	B04	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:51 AM	200C

Make sure you have attached a running cluster to your notebook, and run the entire notebook once before you start working on this notebook further.



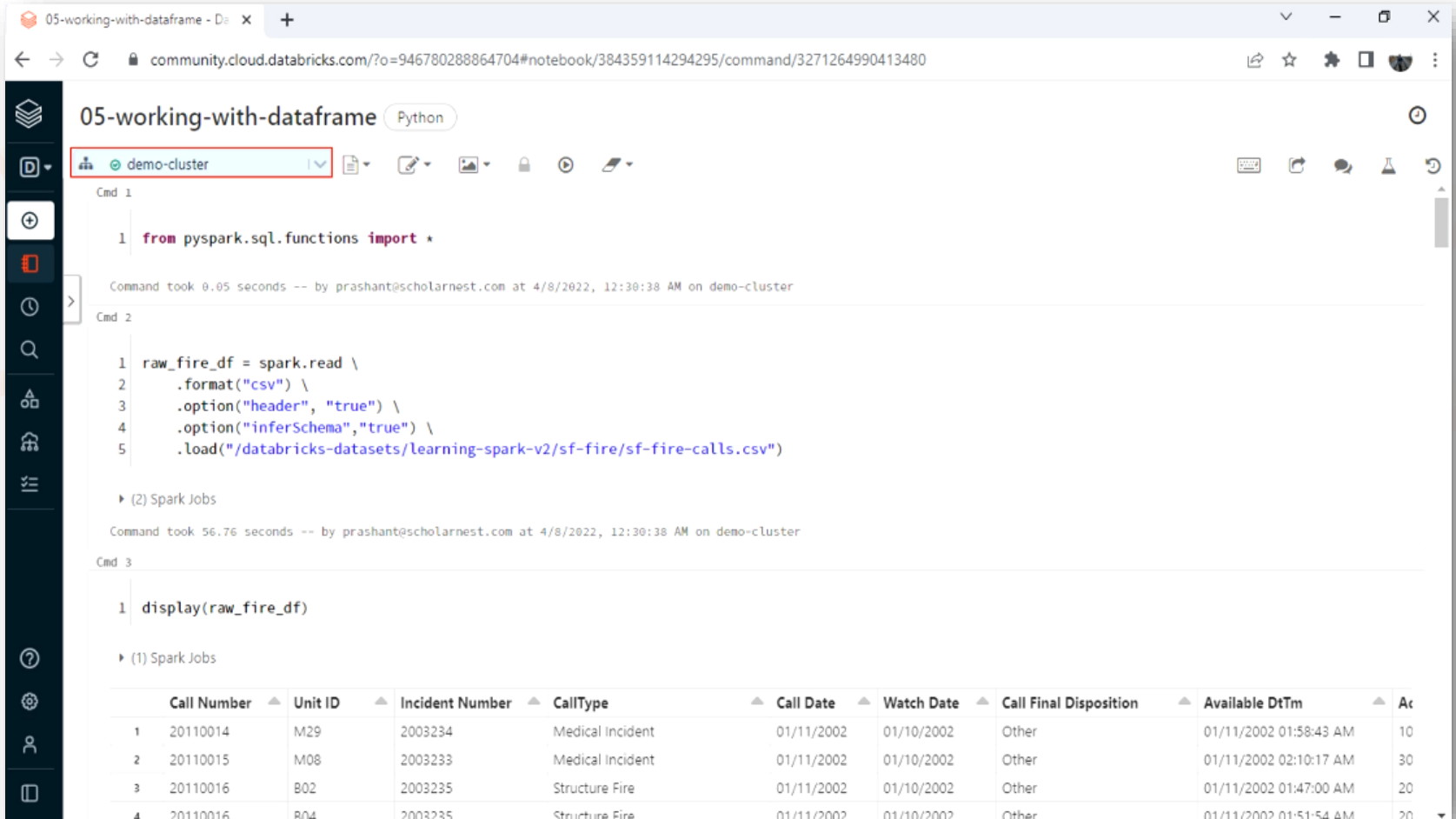
The screenshot shows a Databricks notebook titled "05-working-with-dataframe" in Python. The cluster "demo-cluster" is selected. The notebook contains three commands:

- Cmd 1:** `from pyspark.sql.functions import *`
Command took 0.05 seconds -- by prashant@scholarnest.com at 4/8/2022, 12:30:38 AM on demo-cluster
- Cmd 2:** `raw_fire_df = spark.read \`
`.format("csv") \`
`.option("header", "true") \`
`.option("inferSchema", "true") \`
`.load("/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv")`
(2) Spark Jobs
Command took 56.76 seconds -- by prashant@scholarnest.com at 4/8/2022, 12:30:38 AM on demo-cluster
- Cmd 3:** `display(raw_fire_df)`
(1) Spark Jobs

The output of Cmd 3 is a table with 10 columns: Call Number, Unit ID, Incident Number, CallType, Call Date, Watch Date, Call Final Disposition, Available DtTm, and Ac.

	Call Number	Unit ID	Incident Number	CallType	Call Date	Watch Date	Call Final Disposition	Available DtTm	Ac
1	20110014	M29	2003234	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 01:58:43 AM	10
2	20110015	M08	2003233	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 02:10:17 AM	30
3	20110016	B02	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	20
4	20110016	B04	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:54 AM	20

Make sure you have attached a running cluster to your notebook, and run the entire notebook once before you start working on this notebook further. Whenever you stop working on the Databricks cloud and want to return, you must start a new cluster, attach it to your notebook and recreate all the objects you want to use.



The screenshot shows a Databricks notebook titled "05-working-with-dataframe" in Python. The cluster "demo-cluster" is selected. The notebook contains three commands:

Cmd 1:

```
1 from pyspark.sql.functions import *
```

Command took 0.05 seconds -- by prashant@scholarnest.com at 4/8/2022, 12:30:38 AM on demo-cluster

Cmd 2:

```
1 raw_fire_df = spark.read \
2   .format("csv") \
3   .option("header", "true") \
4   .option("inferSchema", "true") \
5   .load("/databricks-datasets/learning-spark-v2/sf-fire/sf-fire-calls.csv")
```

Command took 56.76 seconds -- by prashant@scholarnest.com at 4/8/2022, 12:30:38 AM on demo-cluster

Cmd 3:

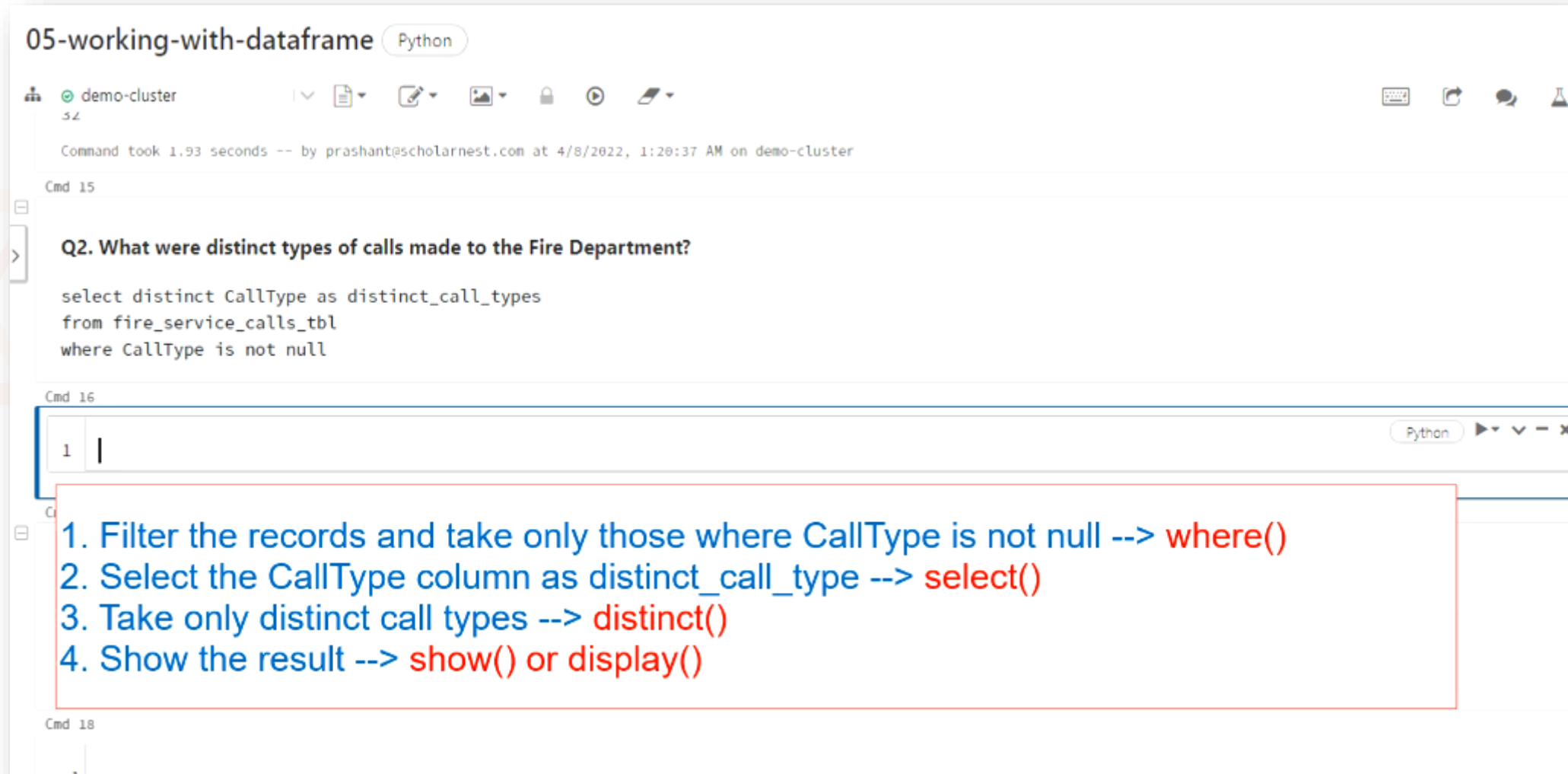
```
1 display(raw_fire_df)
```

Command took 0.05 seconds -- by prashant@scholarnest.com at 4/8/2022, 12:30:38 AM on demo-cluster

The output of Cmd 3 is a table with 10 columns: Call Number, Unit ID, Incident Number, CallType, Call Date, Watch Date, Call Final Disposition, Available DtTm, and Ac. The table contains 4 rows of data.

	Call Number	Unit ID	Incident Number	CallType	Call Date	Watch Date	Call Final Disposition	Available DtTm	Ac
1	20110014	M29	2003234	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 01:58:43 AM	10
2	20110015	M08	2003233	Medical Incident	01/11/2002	01/10/2002	Other	01/11/2002 02:10:17 AM	30
3	20110016	B02	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:47:00 AM	20
4	20110016	B04	2003235	Structure Fire	01/11/2002	01/10/2002	Other	01/11/2002 01:51:54 AM	20

We already solved the first question in the previous lecture. Now, let us look at the second question. We have the logic framed for this question in the image below.



The screenshot shows a Jupyter Notebook titled "05-working-with-dataframe" with a "Python" tab. The notebook is running on a "demo-cluster" environment. A command execution bar shows "Cmd 15" with the following SQL query:

```
Q2. What were distinct types of calls made to the Fire Department?  
  
select distinct CallType as distinct_call_types  
from fire_service_calls_tbl  
where CallType is not null
```

Below the command bar, a red-bordered box highlights the logic for the query:

1. Filter the records and take only those where CallType is not null --> **where()**
2. Select the CallType column as distinct_call_type --> **select()**
3. Take only distinct call types --> **distinct()**
4. Show the result --> **show() or display()**

The bottom of the screenshot shows "Cmd 18" with a cursor in the first cell of a table.

Here is the code for the logic we framed in the second question.

I am using fire_df and applying where transformation. The filter condition takes only those records where CallType is not null. Then I am chaining the select transformation which takes a column name. However, I wanted to give an expression. So, I must use the expr() function to evaluate my expression. Then I am chaining the distinct() transformation. And last step is to take action and show the results.



```
05-working-with-dataframe Python
demo-cluster
3.2
Command took 1.93 seconds -- by prashant@scholarnest.com at 4/8/2022, 1:20:37 AM on demo-cluster

Cmd 15
Q2. What were distinct types of calls made to the Fire Department?

select distinct CallType as distinct_call_types
from fire_service_calls_tbl
where CallType is not null

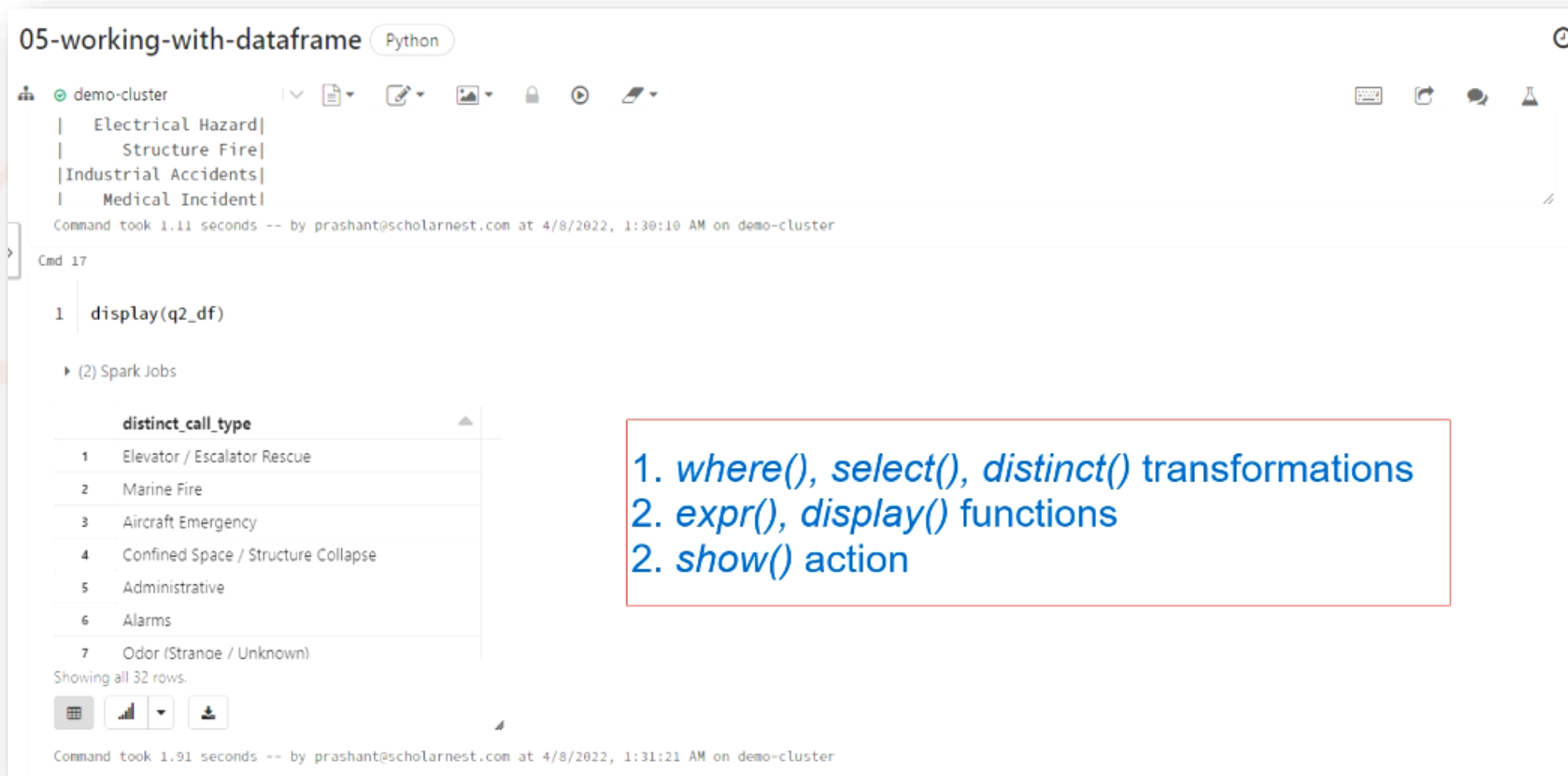
Cmd 16

1 q2_df = fire_df.where("CallType is not null") \
2     .select(expr("CallType as distinct_call_type")) \
3     .distinct()
4 q2_df.show()

▶ (2) Spark Jobs

+-----+
| distinct_call_type|
+-----+
|Elevator / Escala...|
| Aircraft Emergency|
| Alarms|
|Odor (Strange / U...|
|Citizen Assist / ...|
| HazMat|
| Oil Spill|
```

You can also use the `display()` function depicts the formatted results. The `display()` function is not part of Apache Spark. It is a utility function added by Databricks. And the `show()` method is a commonly used Spark action which shows a plain text output. So, we learned the following Dataframe methods from the second question.



The screenshot shows a Databricks notebook titled "05-working-with-dataframe" in Python. The notebook is running on a "demo-cluster". The code cell shows the command `display(q2_df)`. The output is a table with the header "distinct_call_type" and 7 rows of data. The table is displayed in a formatted view. The command took 1.11 seconds to execute. Below the table, it says "Showing all 32 rows." and there are icons for table, bar chart, and download. The command took 1.91 seconds to execute.

	distinct_call_type
1	Elevator / Escalator Rescue
2	Marine Fire
3	Aircraft Emergency
4	Confined Space / Structure Collapse
5	Administrative
6	Alarms
7	Odor (Strandee / Unknown)

Showing all 32 rows.

Command took 1.91 seconds -- by prashant@scholarnest.com at 4/8/2022, 1:31:21 AM on demo-cluster

1. *where(), select(), distinct()* transformations
2. *expr(), display()* functions
2. *show()* action

Now let us look at the 3rd question.

The question asks for all responses for delayed times greater than 5 mins. We have framed the logic for the same in the image shown below.

The screenshot shows a Jupyter Notebook titled "05-working-with-dataframe" with a "Python" language selector. The notebook is running on a "demo-cluster". The command bar shows "Cmd 18" and the command text: "Command took 1.91 seconds -- by prashant@scholarnest.com at 4/8/2022, 1:31:21 AM on demo-cluster".

The main content area displays the following SQL query:

```
select CallNumber, Delay
from fire_service_calls_tbl
where Delay > 5
```

Below the query, a red-bordered box contains the following logic:

- 1. Filter the records where Delay is greater than five --> `where("Delay > 5")`
- 2. Select CallNumber and Delay columns --> `select("CallNumber", "Delay")`

Below the logic box, the command bar shows "Cmd 19" and a table with one row and one column:

1

Below the table, the command bar shows "Cmd 20".

We have implemented the logic into code and we can see the desired output below.

We are doing two new things in this code:

1. We are selecting two columns in the select() method. The select() method takes a comma-separated list of columns.
2. We chained the show() action. Remember that you can chain one and only one Action, and that must be the last method in the chain. So I cannot chain anything else after the show() method.



The screenshot shows a Databricks notebook interface. At the top, the notebook is titled "05-working-with-dataframe" with a "Python" language selector. Below the title, there's a toolbar with various icons. The main area contains a SQL query in a text editor, followed by a command prompt (Cmd 18) and a code cell (Cmd 19). The code cell contains a PySpark query that filters for calls with a delay greater than 5 minutes and selects the call number and delay. The output of the code cell shows a table with 12 rows of data.

```
05-working-with-dataframe Python
```

demo-cluster
Cmd 18

Q3. Find out all response for delayed times greater than 5 mins?

```
select CallNumber, Delay
from fire_service_calls_tbl
where Delay > 5
```

Cmd 19

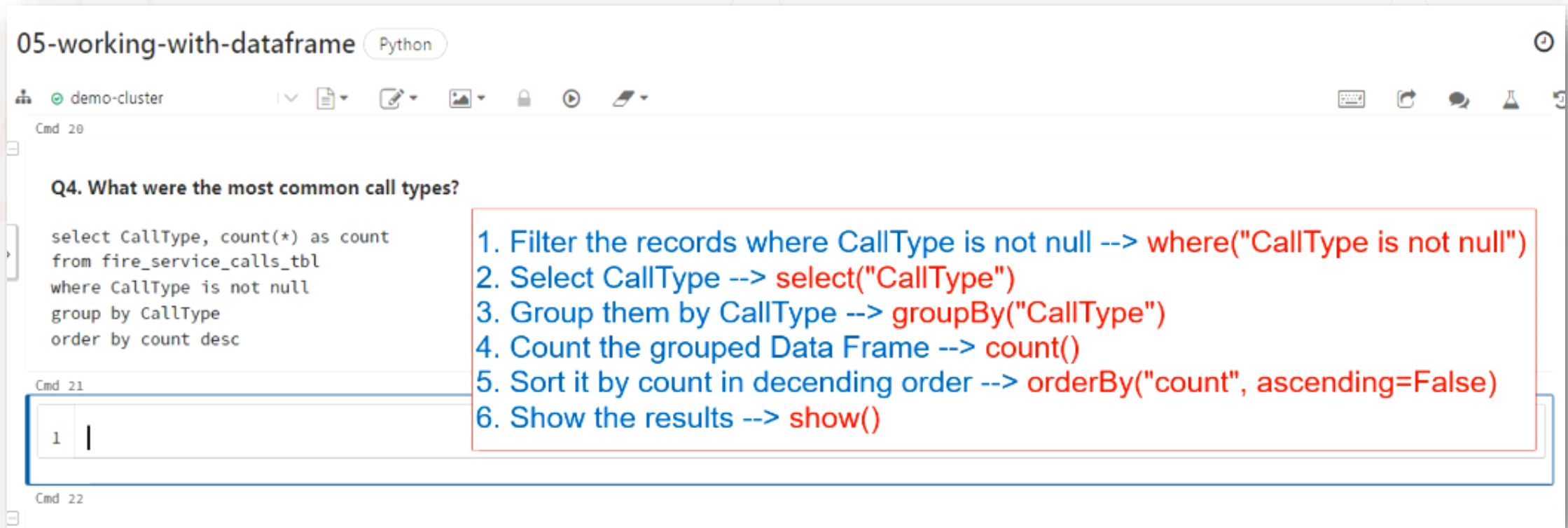
```
1 fire_df.where("Delay > 5") \
2   .select("CallNumber", "Delay") \
3   .show()
```

▶ (1) Spark Jobs

CallNumber	Delay
20110014	5.23
20110017	6.93
20110019	6.12
20110039	7.85
20110045	77.33
20110046	5.42
20110055	6.5
20110058	6.85
20110058	6.85
20110061	6.33

Here we have the next question and it's logic along side.

We want to filter records where the call type is not null and select the CallType. But we want to find the most common CallType. So we must count the calls for each CallType. Hence, we will group by the CallType and then count for each group. The last step is to sort the record by the count in descending order to know which ones are the most common.



The screenshot shows a Jupyter Notebook window titled "05-working-with-dataframe" with a "Python" tab. The interface includes a toolbar with icons for file operations, a "demo-cluster" status indicator, and a command prompt area. The command prompt shows "Cmd 20" and "Cmd 21".

Q4. What were the most common call types?

```
select CallType, count(*) as count
from fire_service_calls_tbl
where CallType is not null
group by CallType
order by count desc
```

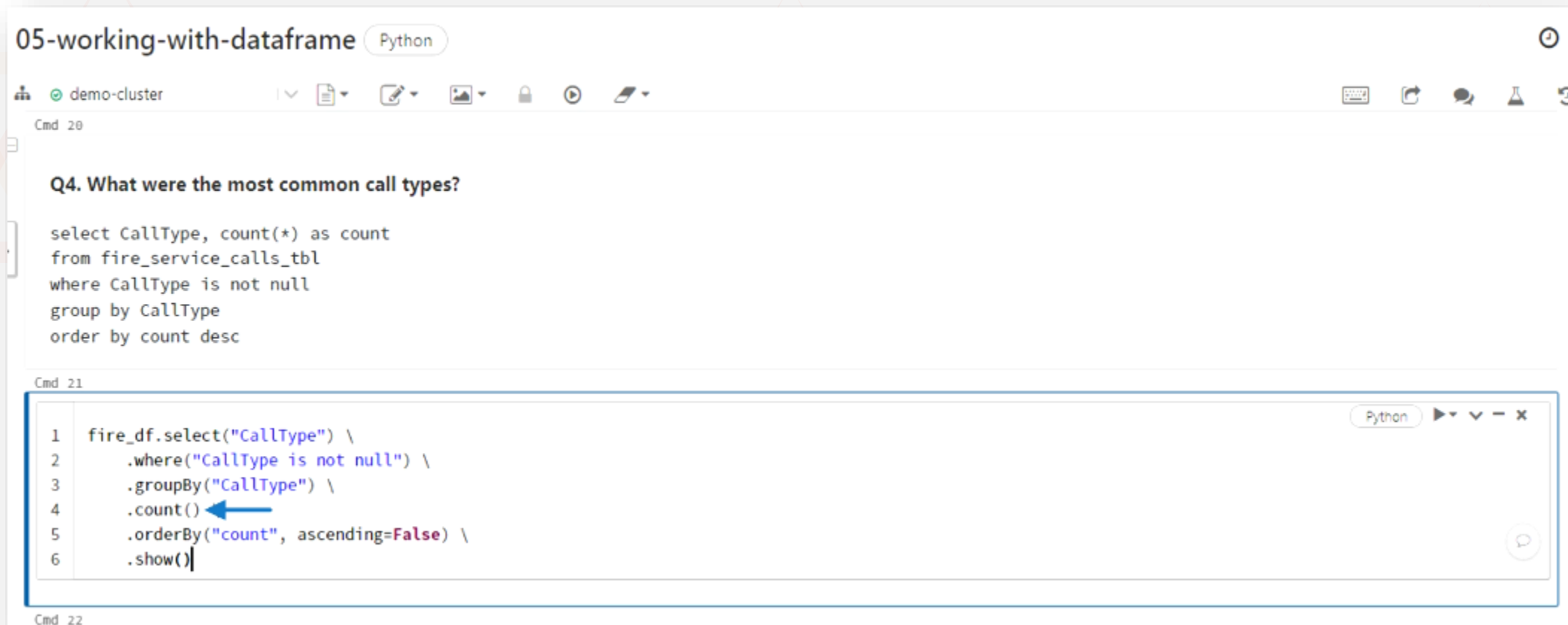
Logic Steps:

1. Filter the records where CallType is not null --> `where("CallType is not null")`
2. Select CallType --> `select("CallType")`
3. Group them by CallType --> `groupBy("CallType")`
4. Count the grouped Data Frame --> `count()`
5. Sort it by count in descending order --> `orderBy("count", ascending=False)`
6. Show the results --> `show()`

The command prompt area shows "Cmd 21" with a table view containing one row with the value "1".

Here is the code for the fourth question.

But I want to highlight one thing here, I am using count() transformation in this example. But we already learned that the count() is an action.



The screenshot shows a Databricks workspace interface. At the top, the tab is labeled '05-working-with-dataframe' with a 'Python' language selector. Below the toolbar, the command history shows 'Cmd 20' with a SQL query: `select CallType, count(*) as count from fire_service_calls_tbl where CallType is not null group by CallType order by count desc`. Below this, 'Cmd 21' contains a PySpark code block with six lines. A blue arrow points to the `.count()` method on line 4. The code is:

```
1 fire_df.select("CallType") \
2     .where("CallType is not null") \
3     .groupBy("CallType") \
4     .count()
5     .orderBy("count", ascending=False) \
6     .show()
```

 The interface includes a 'Python' button and standard window controls (play, close, maximize, minimize) in the top right of the code editor. A chat icon is visible in the bottom right corner.

```
05-working-with-dataframe Python
```

demo-cluster

Cmd 20

Q4. What were the most common call types?

```
select CallType, count(*) as count
from fire_service_calls_tbl
where CallType is not null
group by CallType
order by count desc
```

Cmd 21

```
1 fire_df.select("CallType") \
2     .where("CallType is not null") \
3     .groupBy("CallType") \
4     .count()
5     .orderBy("count", ascending=False) \
6     .show()
```

Cmd 22

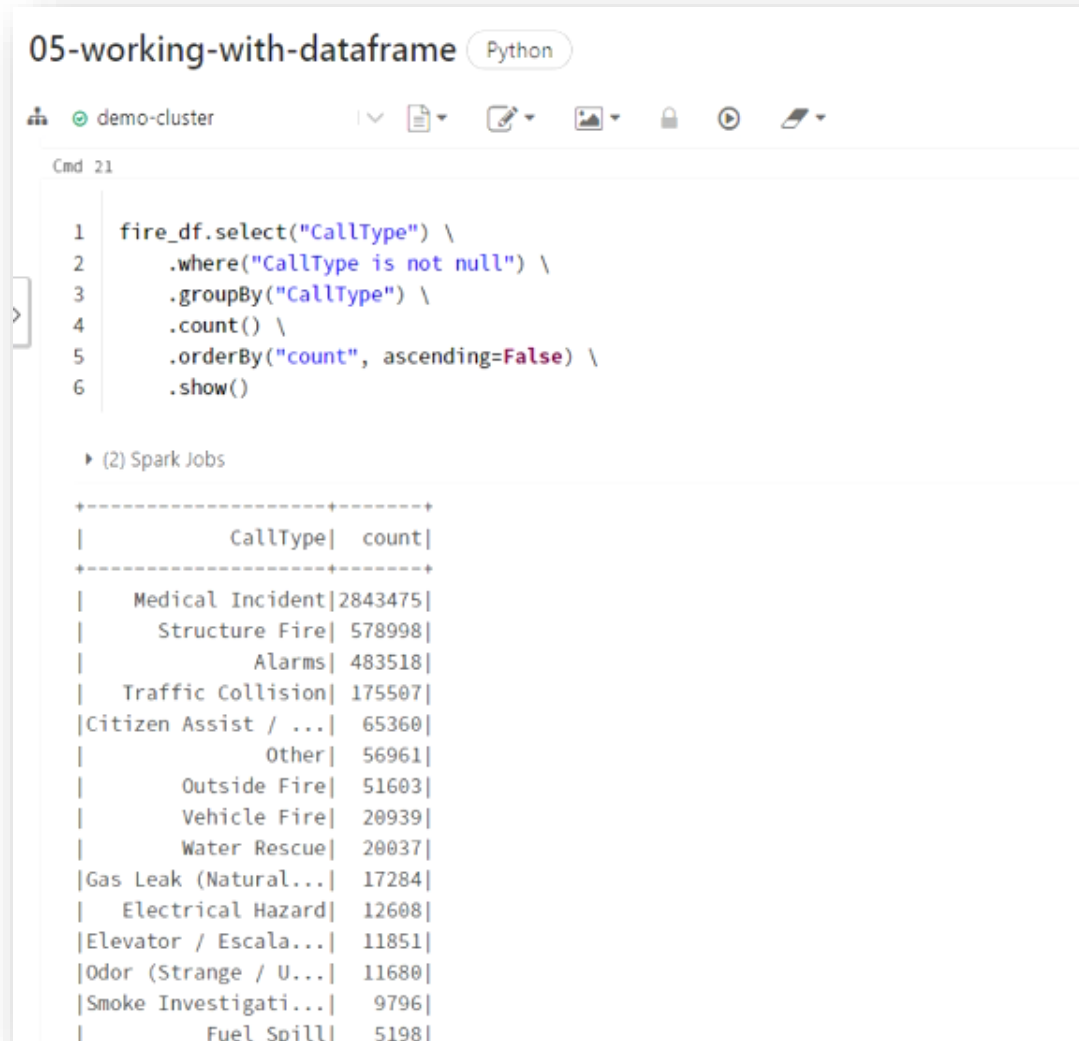
So we have two count() methods in Spark:

1. `Dataframe.count()`
2. `GroupedData.count()`

If you are using a count before the `groupBy()`, it will be an action, and you cannot chain any other transformation after the count () Action.

However, if you use a count() method after the `groupBy()`, it will be a transformation, and you can chain further transformations.

We can see the desired output for the fourth question as well. The remaining 6 questions are given for practice, you can compare your answer with the solutions in the material provided.



```
05-working-with-dataframe Python
demo-cluster
Cmd 21

1 fire_df.select("CallType") \
2   .where("CallType is not null") \
3   .groupBy("CallType") \
4   .count() \
5   .orderBy("count", ascending=False) \
6   .show()

(2) Spark Jobs

+-----+-----+
| CallType | count |
+-----+-----+
| Medical Incident | 2843475 |
| Structure Fire | 578998 |
| Alarms | 483518 |
| Traffic Collision | 175507 |
| Citizen Assist / ... | 65360 |
| Other | 56961 |
| Outside Fire | 51603 |
| Vehicle Fire | 20939 |
| Water Rescue | 20037 |
| Gas Leak (Natural... | 17284 |
| Electrical Hazard | 12608 |
| Elevator / Escala... | 11851 |
| Odor (Strange / U... | 11680 |
| Smoke Investigati... | 9796 |
| Fuel Spill | 5198 |
```



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com